N2S Eval — CLI Walkthrough

Implement sketches + optional demo runner (no browser)

cxr-n2s-eval-workbench/walkthrough

September 2026

Contents

How to use this PDF

Goal: Learn the N2S Eval stack **without the GUI** — same idea as the grounding PDF's Intuition → Minimal pattern → Send-back.

Section	Purpose
Minimal pattern	Short **Python** you should be able to rewrite / whiteboard
Demo runner	Optional `walk_n2s.py` one-liners that call the same APIs

This PDF is the CLI walkthrough (`cxr-n2s-eval-workbench/walkthrough/`).

Not this PDF: `docs/N2S-Workbench-Newcomer-Tour.pdf` (theory + screenshots + SAE UI), `cxr-mi-repeng-grounding/` (foundations ladder), or the portfolio curriculum PDF.

Ladder:

```
W00–W01   Path setup + symbolic evaluate_rule (no LLM)
W02–W04   evaluate_fast / Dual / N2S mismatch
W05       run_forensics_live (observe L20)
W06–W07   run_intervene_live + HF steer hook
W08       Claim hygiene + run_workbench_sae sketch
```

Demo runner only (optional):

```
cd cxr-n2s-eval-workbench
python3 walk_n2s.py list
python3 walk_n2s.py run W02 --model qwen2.5-coder:32b
python3 walk_n2s.py run W06 --alpha 8
```

Prefer HF phases with `../cxrlabs/faiss_gpu1/bin/python`.

Send-backs: `W00 done` … `W08 done`.

N2S Eval — CLI walkthrough

Visitor walkthrough of the project — terminal only, no browser.

Same phases as the UI on port 8257 (CLI covers through Intervene; SAE is in the newcomer tour / lab):

```
Evaluate  →  Deep dive  →  Intervene (alpha)
Track A      observe L20     causal steer + controls
```

Quick start

```
cd cxr-n2s-eval-workbench

python3 walk_n2s.py list
python3 walk_n2s.py show W00
python3 walk_n2s.py run W02 --model qwen2.5-coder:32b
python3 walk_n2s.py run W06 --alpha 8
python3 walk_n2s.py tour --through W04
```

Mock Evaluate (no Ollama):

```
python3 walk_n2s.py run W02 --model mock
```

Layout

Path	Role
`walk_n2s.py`	CLI entry
`walkthrough/lessons/W00…W08.md`	Lesson text: Intuition → Minimal pattern (Python) → optional Demo runner
`walkthrough/lessons.json`	Machine-readable actions / expects
`walkthrough/notes.json`	Bound clinical notes
`docs/N2S-Workbench-Newcomer-Tour.pdf`	Background theory + screenshots
`./build-walkthrough-pdf.sh`	Build CLI walkthrough PDF (tango / Shaded like grounding)
`./build-newcomer-tour-pdf.sh`	Build theory + screenshots PDF (incl. SAE)

Requirements

Lessons	Needs
W00–W01, W08	Nothing
W02–W04	Ollama (or `--model mock`)
W05	Ollama + ~14 GiB free for HF 7B forensics
W06–W07	~14 GiB free for HF 7B intervene

Sibling lab must sit at `../cxr-evidence-grounding-lab`.

Not this track

- `cxr-mi-repeng-grounding/` — long foundations ladder
- `cxr-repeng-curriculum/` — portfolio / interview PDF
- Browser UI (`python3 server.py`) — optional; CLI is the walkthrough path

ROADMAP — N2S CLI walkthrough

```
W00  Welcome (orient)
W01  Atoms + two traps (theory)
W02  Evaluate true contradiction
W03  Evaluate planned → NOT_SATISFIED
W04  Evaluate temporal → often REVIEW
W05  Deep dive L20 + NOT EVALUATED rule
W06  Intervene control (no flip on true contra)
W07  Intervene selective flip + controls
W08  Claim hygiene
```

Visitor path: **W00 → W04** (Evaluate story) then **W05 → W07** if GPU free.

Build PDF: `./build-walkthrough-pdf.sh` → `docs/N2S-CLI-Walkthrough.pdf`

W00 — Welcome

Track: N2S Eval CLI walkthrough · **Next:** W01

Intuition

Someone asks: *walk me through the project* **and** *could you implement this without the GUI?*

- **Demo runner** `walk_n2s.py` — one command that exercises the real stack (visitor path).
- **Minimal pattern** (from W01 on) — short Python you should be able to rewrite: Dual / gates, L20 forensics, h ← h + α·v, SAE top-k.

This tree is the **demo + implementer** walkthrough. It is not the long MI foundations ladder (`cxr-mi-repeng-grounding/`) and not the portfolio PDF (`cxr-repeng-curriculum/`).

```
Evaluate  →  Deep dive  →  Intervene (alpha)  [+ SAE in tour / W08 sketch]
Track A      observe L20     causal steer + controls
```

Demo runner (optional)

```
cd cxr-n2s-eval-workbench
python3 walk_n2s.py list
python3 walk_n2s.py show W00
python3 walk_n2s.py run W02 --model qwen2.5-coder:32b
python3 walk_n2s.py run W06 --alpha 8
```

Minimal pattern — path setup (every later snippet)

Run from `cxr-n2s-eval-workbench/`. Lab code lives one directory up.

```
import sys
from pathlib import Path

ROOT = Path(".").resolve()                # cxr-n2s-eval-workbench
LAB = ROOT.parent / "cxr-evidence-grounding-lab"
sys.path[:0] = [str(ROOT), str(LAB)]
```

Prefer HF GPU phases with `../cxrlabs/faiss_gpu1/bin/python` (not bare system Python).

Companion reading

- Theory + screenshots + SAE UI: `docs/N2S-Workbench-Newcomer-Tour.pdf`
- Frozen α=8 claim: `../cxr-evidence-grounding-lab/docs/TRACKB-ALPHA8-FREEZE.md`

Send-back

W00 done — one sentence: demo runner vs Minimal pattern (where the learning is).

W01 — Atoms and two traps

Prerequisite: W00 · **Next:** W02

Intuition

`FIRST_LINE_THERAPY_FAILED` — did first-line therapy fail?

Atom	Meaning
A	First-line therapy identified
B	Actually given (not only planned)
C	Failure / progression event
D	Failure of that first-line
X	Same-time incompatible claims → CONTRADICTION

Two traps

1. **True contradiction** — same visit: failed *and* still responding.
2. **Temporal change** — responded earlier, progressed later. Gold often **SATISFIED**, not contradiction. Models may wrongly set **X=true**.

N2S in one line: neural extract → symbolic `ground` → Dual paths C/D → gates → AUTO or REVIEW.

The **neural** side fills atoms. The **symbolic** rule alone decides the verdict — no LLM required for this lesson.

Demo runner (optional)

```
python3 walk_n2s.py show W01
```

Minimal pattern — symbolic rule (no LLM)

```
import sys
from pathlib import Path

LAB = Path("../cxr-evidence-grounding-lab").resolve()
sys.path.insert(0, str(LAB))

from n2s_lab.types import Atom, Grounding
from n2s_lab.predicate import evaluate_rule

# Planned only → B false → NOT_SATISFIED
planned = Grounding(
    first_line_identified=Atom.TRUE,
    first_line_administered=Atom.FALSE,  # B
    failure_event=Atom.UNKNOWN,
    failure_of_first_line=Atom.UNKNOWN,
    contradiction=False,
)
print(evaluate_rule(planned).verdict)  # NOT_SATISFIED

# Same-day conflict → X → CONTRADICTION (short-circuit)
contra = Grounding(
    first_line_identified=Atom.TRUE,
    first_line_administered=Atom.TRUE,
    failure_event=Atom.TRUE,
    failure_of_first_line=Atom.TRUE,
    contradiction=True,  # X
)
print(evaluate_rule(contra).verdict)  # CONTRADICTION

# Temporal success story atoms (no X) → SATISFIED
temporal = Grounding(
    first_line_identified=Atom.TRUE,
    first_line_administered=Atom.TRUE,
    failure_event=Atom.TRUE,
    failure_of_first_line=Atom.TRUE,
    contradiction=False,
)
print(evaluate_rule(temporal).verdict)  # SATISFIED
```

Or: `python3 -c "from n2s_lab.predicate import selftest; selftest()"` from the lab root.

Send-back

W01 done — why temporal change is not a contradiction, and what `evaluate_rule` returns for B=false vs X=true.

W02 — Evaluate: true contradiction

Prerequisite: W01 · **Next:** W03 · **Action:** `evaluate` · **Note:** `EX_CONTRA`

Intuition

Same-day conflict should AUTO as CONTRADICTION when Dual and grounding agree.

Track A wiring (what you implement / call):

```
note → analyze → path C extract + path D extract
    → ground / verify / evidence → dual_path_verdict
    → N2S mismatch (neural X vs grounded X) → apply_gates → AUTO|REVIEW
```

Demo runner (optional)

```
python3 walk_n2s.py run W02 --model qwen2.5-coder:32b
# dry wiring:  python3 walk_n2s.py run W02 --model mock
```

Minimal pattern — Evaluate without the GUI

```
import sys
from pathlib import Path

ROOT = Path(".").resolve()  # cxr-n2s-eval-workbench
sys.path[:0] = [str(ROOT), str(ROOT.parent / "cxr-evidence-grounding-lab")]

import eval_api

NOTE = (
    "Note A: Patient failed first-line FOLFOX after four cycles. "
    "Addendum same day: Disease remains responsive to FOLFOX; continue current regimen."
)

fast = eval_api.evaluate_fast(
    {
        "note": NOTE,
        "gold": "CONTRADICTION",
        "case_id": "SKETCH_CONTRA",
        "model": "qwen2.5-coder:32b",  # or "mock"
        "unload_after": True,
    }
)

print(fast["disposition"], fast["verdict"])
print("mismatch agree:", (fast.get("mismatch") or {}).get("agree"))
print("paths C/D:", fast["paths"]["C_full"]["verdict"], fast["paths"]["D_full"]["verdict"])
```

Expect roughly:

```
AUTO CONTRADICTION
mismatch agree: True
```

Core idea — Dual + mismatch (whiteboard)

```
from n2s_lab.roundtrip import dual_path_verdict

# After both paths have a symbolic verdict string:
dual = dual_path_verdict(verdict_c, verdict_d)
# dual["disposition"] is AUTO only if C and D agree on a non-REVIEW verdict

# N2S mismatch (simplified — see eval_api._mismatch):
neural_x = extraction["contradiction"]["present"]   # extractor claim
grounded_x = grounding["contradiction"]             # after ground()
if neural_x is True and grounded_x is False:
    disposition = "REVIEW"  # classic temporal false-X containment
```

Send-back

W02 done — paste disposition + verdict, and name the two functions Dual vs mismatch use.

W03 — Evaluate: planned only

Prerequisite: W02 · **Next:** W04 · **Action:** `evaluate` · **Note:** `EX_PLANNED`

Intuition

Therapy planned next week, none given yet → atom **B** false → **NOT_SATISFIED**.

Same `evaluate_fast` entry as W02; only the note changes. Deep dive later marks representation–commitment **NOT EVALUATED** when commitment is NOT_SATISFIED (W05).

Demo runner (optional)

```
python3 walk_n2s.py run W03
```

Minimal pattern — same API, planned note

```
import sys
from pathlib import Path

ROOT = Path(".").resolve()
sys.path[:0] = [str(ROOT), str(ROOT.parent / "cxr-evidence-grounding-lab")]
import eval_api

NOTE = (
    "Newly diagnosed metastatic colon cancer. Multidisciplinary plan is to start "
    "first-line FOLFOX next week. No systemic therapy has been administered yet."
)

fast = eval_api.evaluate_fast(
    {
        "note": NOTE,
        "gold": "NOT_SATISFIED",
        "case_id": "SKETCH_PLANNED",
        "model": "qwen2.5-coder:32b",
        "unload_after": True,
    }
)
print(fast["disposition"], fast["verdict"])
# Typical: AUTO NOT_SATISFIED  (B false after ground → evaluate_rule)
```

Send-back

W03 done — paste disposition + verdict; name which atom fails.

W04 — Evaluate: temporal containment

Prerequisite: W03 · **Next:** W05 · **Action:** `evaluate` · **Note:** `EX_TEMPORAL_FOLFOX`

Intuition

Clinically: response then later progression → gold **SATISFIED**, not contradiction.

Track A often **REVIEW**s when the extractor sets neural X=true while `ground` clears X as temporal-change (**N2S mismatch**). That is containment, not product failure.

Demo runner (optional)

```
python3 walk_n2s.py run W04
```

Minimal pattern — read mismatch explicitly

```
import sys
from pathlib import Path

ROOT = Path(".").resolve()
sys.path[:0] = [str(ROOT), str(ROOT.parent / "cxr-evidence-grounding-lab")]
import eval_api

NOTE = (
    "Metastatic colorectal cancer. First-line FOLFOX produced a partial response. "
    "At follow-up four months later, imaging demonstrated new hepatic lesions "
    "consistent with progression. FOLFOX was discontinued for treatment failure; "
    "second-line therapy discussed."
)

fast = eval_api.evaluate_fast(
    {
        "note": NOTE,
        "gold": "SATISFIED",
        "case_id": "SKETCH_TEMPORAL",
        "model": "qwen2.5-coder:32b",
        "unload_after": True,
    }
)

mm = fast.get("mismatch") or {}
print(fast["disposition"], fast["verdict"])
print("neural_x", mm.get("neural_contradiction"), "grounded_X", mm.get("grounded_X"))
print("agree", mm.get("agree"), mm.get("reasons"))
# Soft expect: REVIEW + agree False  OR  AUTO SATISFIED if X already clean
```

Teaching point

Do not patch every paste to force AUTO. Investigate Dual / mismatch; REVIEW is OK.

Send-back

W04 done — paste headline + neural_x / grounded_X / agree.

W05 — Deep dive: observe L20

Prerequisite: W04 · **Next:** W06 · **Action:** `deep_dive` · **Note:** `EX_PLANNED`

Intuition

Deep dive **observes**. Round-trip checks + HF Qwen 7B layer scores (L20 temporal-change vs contradiction). It does **not** change the model.

Label rule: agree/mismatch comparison only when commitment is **CONTRADICTION**.
For **NOT_SATISFIED** / **SATISFIED** → **NOT EVALUATED**.

Needs ~14 GiB free VRAM (unload Ollama first).

Demo runner (optional)

```
python3 walk_n2s.py run W05
```

Minimal pattern — forensics live (HF hooks under the hood)

```
import sys
from pathlib import Path

ROOT = Path(".").resolve()
sys.path[:0] = [str(ROOT), str(ROOT.parent / "cxr-evidence-grounding-lab")]

import eval_api
from n2s_lab.n2s_forensics import format_forensics_text, run_forensics_live
from n2s_lab.ollama_client import unload_model

NOTE = (
    "Newly diagnosed metastatic colon cancer. Multidisciplinary plan is to start "
    "first-line FOLFOX next week. No systemic therapy has been administered yet."
)

# 1) Track A first (optional but matches UI)
fast = eval_api.evaluate_fast(
    {
        "note": NOTE,
        "gold": "NOT_SATISFIED",
        "case_id": "SKETCH_DD",
        "model": "qwen2.5-coder:32b",
        "unload_after": True,
    }
)
print(fast["disposition"], fast["verdict"])

# 2) Free Ollama VRAM, then HF L20 readout
unload_model("qwen2.5-coder:32b")

panel = run_forensics_live(
    NOTE,
    case_id="SKETCH_DD",
    gold="NOT_SATISFIED",
    final_commitment=fast["verdict"],
    unload_after=True,
)
print(panel.get("status") or panel.get("ok"))
print(format_forensics_text(panel) if panel.get("ok") else panel.get("message"))
# Look for L20 soft scores + "NOT EVALUATED" when commitment is NOT_SATISFIED
```

Workbench UI path is the same idea via `eval_api._deep_dive_work(fast)` (round-trip + forensics).

Send-back

W05 done — paste L20 soft scores (or deferred_gpu) + comparison line.

W06 — Intervene: control (no flip)

Prerequisite: W05 · **Next:** W07 · **Action:** `intervene` · **Note:** `EX_CONTRA` · **alpha:** 8

Intuition

Steer `h ← h + alpha · v` at L20 on the `contradiction.present` commit token. True contradictions must **stay** X=true.

Arms: baseline, full_vector, gaussian_unit, reverse_vector.

Demo runner (optional)

```
python3 walk_n2s.py run W06 --alpha 8
```

Minimal pattern — expand editor API

```
import sys
from pathlib import Path

ROOT = Path(".").resolve()
sys.path[:0] = [str(ROOT), str(ROOT.parent / "cxr-evidence-grounding-lab")]

from n2s_lab.n2s_intervene import format_intervene_text, run_intervene_live

NOTE = (
    "Note A: Patient failed first-line FOLFOX after four cycles. "
    "Addendum same day: Disease remains responsive to FOLFOX; continue current regimen."
)

panel = run_intervene_live(
    NOTE,
    case_id="SKETCH_CONTRA",
    gold="CONTRADICTION",
    alpha=8.0,
    include_controls=True,
    unload_after=True,
)
print(format_intervene_text(panel) if panel.get("ok") else panel)
# Expect: baseline X=true and full_vector X=true (no true→false flip)
for arm in panel.get("arms") or []:
    print(arm["label"], "X=", arm.get("repair_final_x"), "margin=", arm.get("margin"))
```

Core idea — HF forward hook (what the library does)

```
# Sketch of n2s_lab/hf_intervene.py activation_steer (Qwen: model.model.layers[i])
def steer_hook(_module, _inp, out, *, vec, alpha):
    hs = out[0] if isinstance(out, tuple) else out
    modified = hs.clone()
    # last position = commit token when steer_active
    modified[0, -1, :] = modified[0, -1, :] + alpha * vec
    return (modified,) + out[1:] if isinstance(out, tuple) else modified

# handle = layers[20].register_forward_hook(...)
# v = unit(mean_A − mean_B) from artifacts/n2s-forensics-directions-qwen7b.json
```

Frozen limited claim @ α=8 — see lab `TRACKB-ALPHA8-FREEZE.md`.

Send-back

W06 done — paste four-arm X / margin; state the hook equation in one line.

W07 — Intervene: selective flip

Prerequisite: W06 · **Next:** W08 · **Action:** `intervene` · **Note:** EX_TEMPORAL_FOLFOX · **alpha:** 8

Intuition

On some temporal false-X notes, baseline X=true → full_vector X=false while gaussian stays true and reverse does not help.

That is the live demo of the limited expand editor. Soft: if baseline already X=false, pick a harder expand fixture (e.g. TX_E14).

Demo runner (optional)

```
python3 walk_n2s.py run W07 --alpha 8
```

Minimal pattern — same API, temporal note + controls

```
import sys
from pathlib import Path

ROOT = Path(".").resolve()
sys.path[:0] = [str(ROOT), str(ROOT.parent / "cxr-evidence-grounding-lab")]

from n2s_lab.n2s_intervene import format_intervene_text, run_intervene_live

NOTE = (
    "Metastatic colorectal cancer. First-line FOLFOX produced a partial response. "
    "At follow-up four months later, imaging demonstrated new hepatic lesions "
    "consistent with progression. FOLFOX was discontinued for treatment failure; "
    "second-line therapy discussed."
)

panel = run_intervene_live(
    NOTE,
    case_id="SKETCH_TEMPORAL",
    gold="SATISFIED",
    alpha=8.0,
    include_controls=True,  # gaussian_unit + reverse_vector
    unload_after=True,
)

print(format_intervene_text(panel) if panel.get("ok") else panel)
by = {a["label"]: a for a in (panel.get("arms") or [])}
for name in ("baseline", "full_vector", "gaussian_unit", "reverse_vector"):
    a = by.get(name) or {}
    print(f"{name:16} X={a.get('repair_final_x')}  margin={a.get('margin')}")

# Credible story: baseline true → full_vector false; gaussian true; reverse not helping
```

Send-back

W07 done — paste arms table; say whether flip happened and what the controls did.

W08 — Claim hygiene (+ SAE sketch)

Prerequisite: W07

Say

- Track A contains uncertain transforms via REVIEW / Dual / mismatch.
- Track B shows a **partial** L20 α=8 editor with controls on expand-style temporal false-X.
- You can demo with `walk_n2s.py` **or** call `evaluate_fast` / `run_forensics_live` / `run_intervene_live` directly.

Do not say

- Steering fixed healthcare / all temporal false-X.
- α=16/32 is the freeze.
- 14B MI editor transfer (behavioral transfer only; cluster did not persist).
- Sealed `temporal-family-test.json` was used for fitting.
- SAE ranking = "we found the temporality neuron."

What is next (optional)

- **SAE features** — Chanin L20 decompose **v**; workbench `:8257` or sketch below.
- Upstream prefill + thinner circuits — lab HF hooks.
- Foundations ladder: `cxr-mi-repeng-grounding/`.
- Portfolio spine: `cxr-repeng-curriculum/`.

Demo runner (optional)

```
python3 walk_n2s.py show W08
python3 walk_n2s.py list
```

Minimal pattern — SAE without the GUI

```
import sys
from pathlib import Path

ROOT = Path(".").resolve()
sys.path[:0] = [str(ROOT), str(ROOT.parent / "cxr-evidence-grounding-lab")]

from n2s_lab.n2s_sae_pilot import run_workbench_sae

NOTE = (
    "Metastatic colorectal cancer. First-line FOLFOX produced a partial response. "
    "At follow-up four months later, imaging demonstrated new hepatic lesions "
    "consistent with progression. FOLFOX was discontinued for treatment failure."
)

# Prefer: ../cxrlabs/faiss_gpu1/bin/python
panel = run_workbench_sae(
    note=NOTE,
    case_id="SKETCH_SAE",
    gold="SATISFIED",
    top_k=3,
    alpha=8.0,
    include_contra_control=True,  # EX_CONTRA must not flip true→false
)
# Ranked feats: Δ(A−B), cos_to_v, track X/margin — then top-k steers
for row in (panel.get("candidates") or [])[:5]:
    print(row.get("feat_id"), row.get("delta_A_minus_B"), row.get("cos_to_v"))
```

Lab docs: `docs/TRACKB-SAE-PILOT.md`. Ranking ≠ causal proof; read the arms table.

Send-back

W08 done — one sentence claim you would tell a visitor (Track A + limited α=8 + SAE optional).